

</ Pole a textové řetězce v programování />

Dominik Němeček

Předmět programování

} /> [

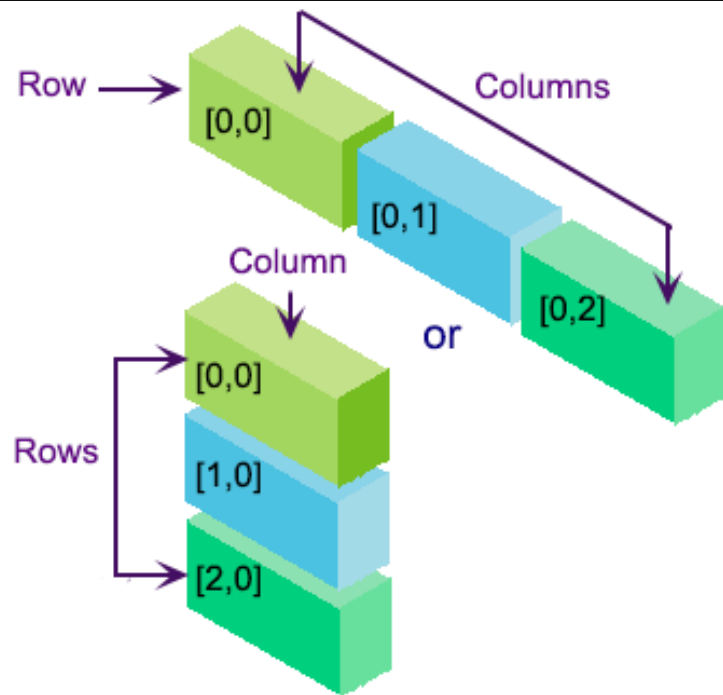
</ Pole />

- Pole je **datová struktura sloužící k ukládání a organizaci více hodnot** stejného datového typu pod jedním názvem.
- Je to **seznam prvků, který je v paměti uložen za sebou** a je indexován od 0.
- Musíme si hlídat velikost pole (pracujeme s ukazatelem)
 - **C nehlídá, jestli čísla uložíme mimo pole.**

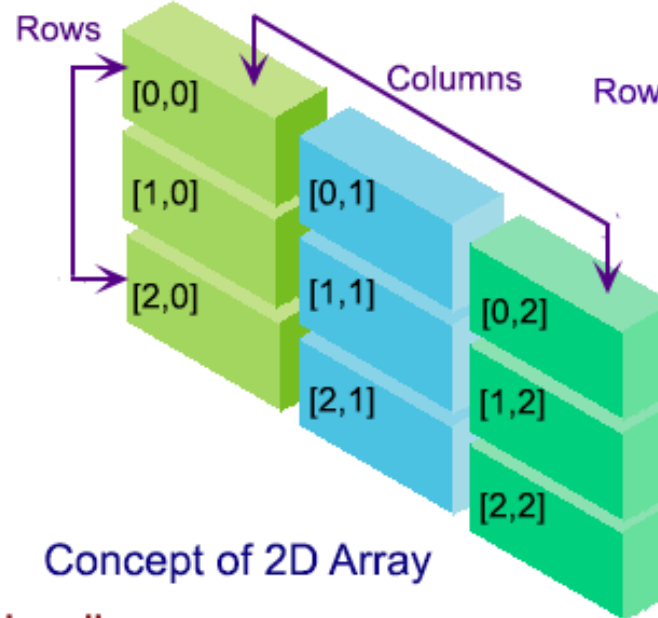
Index	0	1	2	3	4	5
Číslo	10	2	8	-5	7	3

← Pole čísel uložené
v paměti počítače

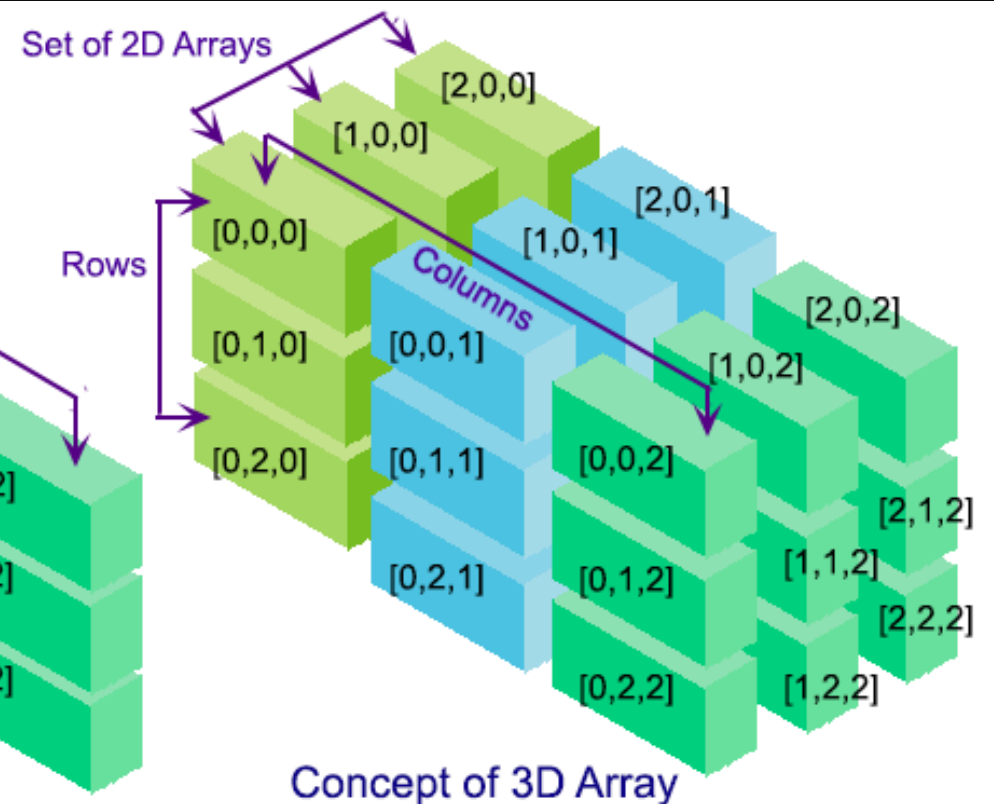
</Pole />



Concept of 1D Array



Concept of 2D Array



Concept of 3D Array

Exercises are explained step-by-step visually.

© w3resource.com

</Využití pole />

- 1) Ukládání seznamu prvků a kolekce dat
- 2) Maticové operace
- 3) Zpracování textu
- 4) Sledování statistik
- 5) Tvorba základních databází

</ Deklarace a inicializace pole />

- **Syntaxe:** (datový typ, název pole, velikost pole)
- První způsob plnění pole: `int pole[6];`
(poté jednotlivě přiřadíme čísla)
- Druhý způsob vytvoření pole: `int pole[]`
`= {10, 2, 8, -5, 7, 3};`

```
1  #include <stdio.h>
2
3  int main ()
4  {
5      // První způsob
6      int pole1[6];
7      pole1[0] = 10;
8      pole1[1] = 2;
9      pole1[2] = 8;
10     pole1[3] = -5;
11     pole1[4] = 7;
12     pole1[5] = 3;
13
14     // Druhý způsob
15     int pole2[] = {10, 2, 8, -5, 7, 3};
16     return 0;
17 }
```

Index	0	1	2	3	4	5
Číslo	10	2	8	-5	7	3

</Implementace pole/>

{Použití pole - ukázka}

- Rozdíl mezi použitím a nepoužitím pole.

```
1  #include <stdio.h>
2
3  int main ()
4  {
5      // Nepoužití pole
6      int vek_uzivatele1 = 15;
7      int vek_uzivatele2 = 18;
8      int vek_uzivatele3 = 25;
9      int vek_uzivatele4 = 32;
10     int vek_uzivatele5 = 16;
11
12     // Použití pole
13     int vek_uzivatelu[] = {15, 18, 25, 32, 16};
14     return 0;
15 }
```

</Implementace pole/>

{Práce s polem – ukázka}

- Naplnění pole 10 čísly (1 až 10) a vypsání pomocí **for** cyklu.

```
1  #include <stdio.h>
2  #define VELIKOST_POLE 10 // Definování konstanty
3
4  int main ()
5  {
6      int pole[VELIKOST_POLE]; // Pole o velikosti 10 intů - později místo čísel využívat proměnné/konstanty
7
8      // Naplnění pole - pozor na indexování pole od 0
9      for (int i = 0; i < VELIKOST_POLE; i++)
10     {
11         pole[i] = i + 1; // Indexování od 0 - přičteme +1, abychom začínali od 1
12         printf("%d ", pole[i]); // Výpis hodnot v poli
13     }
14
15     return 0;
16 }
```

</Implementace pole />

{Součet prvků v poli – ukázka}

- Požádáme uživatele, aby zadal 5 čísel, která následovně uložíme do pole.
- Nakonec vypíšeme součet všech čísel v poli.

```
Zadejte 5 celych cisel:
1. cislo: 2
2. cislo: 2
3. cislo: 1
4. cislo: 3
5. cislo: 1
Soucet cisel v poli je: 9
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int ciska[5];
6      int soucet = 0;
7
8      printf("Zadejte 5 celych cisel:\n");
9      for (int i = 0; i < 5; i++)
10     {
11         printf("%d. cislo: ", i + 1);
12         scanf("%d", &ciska[i]);
13         soucet += ciska[i];
14     }
15
16     printf("Soucet cisel v poli je: %d\n", soucet);
17
18     return 0;
19 }
```

← Ukázka výstupu

</ Pole s délkou, kterou určíme za běhu & určení velikosti pole />

{Práce s polem – ukázka}

- Standard C99 umožňuje deklarovat tzv. VLA (Variable Length Array) – pole s délkou, kterou zadáme až za běhu programu.

```
1  #include <stdio.h>
2
3  int main ()
4  {
5      int velikost_pole;
6      printf("Zadejte velikost pole pro vytvoreni: \n");
7      scanf("%d", &velikost_pole);
8      int pole[velikost_pole]; // Vytvoření pole o velikosti zadané od uživatele
9      printf("Pocet prvku pole: %dx int\n", sizeof(pole)/sizeof(int)); // Zjištění počtu prvků v poli
10     printf("Velikost pole v pameti: %d bytes\n", sizeof(pole)); // Zjištění velikosti pole v bytes
11     return 0;
12 }
```

</ Dvourozměrné pole />

- Doted' jsme využívali jednorozměrné pole.
- Dvourozměrné pole si **můžeme představit jako tabulku.**

pole[][počet sloupců]

Index	0	1	2	3	4
0	10	2	8	-5	7
1	3	4	3	8	15
2	13	142	34	48	153
3	4	86	59	74	35
4	0	1	9	23	29

pole[počet řádků][]

```
1  #include <stdio.h>
2
3  int main ()
4  {
5      // První způsob
6      int pole[5][5]; // int pole[řádky][sloupce];
7      pole[0][0] = 10;
8      pole[0][1] = 2;
9      //pole[][]...
10
11     // Druhý způsob
12     int pole[5][5] =
13     {
14         {10, 2, 8, -5, 7},
15         {3, 4, 3, 8, 15},
16         {13, 142, 34, 48, 153},
17         {4, 86, 59, 74, 35},
18         {0, 1, 9, 23, 29}
19     };
20     return 0;
21 }
```

</Implementace dvourozměrného pole/>

{Malá násobilka - ukázka}

- Naplníme pole 100 čísly (1 až 100) a vypíšeme pomocí **for** cyklu.
- (Pozn. Pozor na překročení mezí pole!)

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

```
1  #include <stdio.h>
2  #define POCET_RADKU 10
3  #define POCET_SLOUPCU 10
4
5  int main ()
6  {
7      int pole[POCET_RADKU][POCET_SLOUPCU]; // Vytvoření pole o velikosti 10x10
8
9      for (int i = 0; i < POCET_RADKU; i++)
10     {
11         for (int j = 0; j < POCET_SLOUPCU; j++)
12         {
13             pole[i][j] = (i + 1) * (j + 1); // Ukládání hodnot do pole
14             printf("%4d ", pole[i][j]); // Vypsání pole
15         }
16         printf("\n");
17     }
18
19     return 0;
20 }
```

← Ukázka výstupu

</Implementace pole />

{Cvičení 01 – Největší a nejmenší prvek}

- Požádáme uživatele, aby zadal délku pole (poté zadá jednotlivé prvky).
- Nakonec vypíšeme největší a nejmenší prvek.

```
Zadejte delku pole: 4
Zadejte prvky pole:
Prvek 1: 1
Prvek 2: 2
Prvek 3: 3
Prvek 4: 4
Nejmensi prvek v poli je: 1
Nejvetsi prvek v poli je: 4
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int delka_pole;
6      printf("Zadejte delku pole: ");
7      scanf("%d", &delka_pole);
8      int pole[delka_pole];
9      printf("Zadejte prvky pole:\n");
10     for (int i = 0; i < delka_pole; i++) // Zadávání prvků do pole
11     {
12         printf("Prvek %d: ", i + 1);
13         scanf("%d", &pole[i]);
14     }
15     int nejmensi = pole[0];
16     int nejvetsi = pole[0];
17     for (int i = 1; i < delka_pole; i++) // Porovnávání prvků v poli
18     {
19         // Pokud je prvek pole menší, než proměnná nejmensi = přepíšeme proměnnou nejmensi
20         if (pole[i] < nejmensi)
21         {
22             nejmensi = pole[i];
23         }
24         // Pokud je prvek pole větší, než proměnná nejvetsi = přepíšeme proměnnou nejvetsi
25         else if (pole[i] > nejvetsi)
26         {
27             nejvetsi = pole[i];
28         }
29     }
30     printf("Nejmensi prvek v poli je: %d\n", nejmensi);
31     printf("Nejvetsi prvek v poli je: %d\n", nejvetsi);
32     return 0;
33 }
```

← Ukázka výstupu

</Implementace pole />

{Cvičení 02 – Hledání prvku v poli}

- Požádáme uživatele, aby zadal délku pole (poté zadá jednotlivé prvky).
- Nakonec uživatel zadá hledaný prvek a my vypíšeme pozici prvku.

```
Zadejte velikost pole: 3
Zadejte prvky pole:
Prvek 1: 1
Prvek 2: 8
Prvek 3: 4
Zadejte prvek, který chcete najít: 8
Prvek 8 byl nalezen na pozici 2.
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int velikost_pole;
6      printf("Zadejte velikost pole: ");
7      scanf("%d", &velikost_pole);
8      int pole[velikost_pole];
9      printf("Zadejte prvky pole:\n");
10     for (int i = 0; i < velikost_pole; i++)
11     {
12         printf("Prvek %d: ", i + 1);
13         scanf("%d", &pole[i]);
14     }
15     int hledany_prvek;
16     printf("Zadejte prvek, který chcete najít: ");
17     scanf("%d", &hledany_prvek);
18     int nalezen = 0;
19     int index;
20     for (int i = 0; i < velikost_pole; i++)
21     {
22         if (pole[i] == hledany_prvek)
23         {
24             nalezen = 1;
25             index = i;
26         }
27     }
28     if (nalezen == 1)
29     {
30         printf("Prvek %d byl nalezen na pozici %d.\n", hledany_prvek, index + 1);
31     }
32     else
33     {
34         printf("Prvek %d nebyl v poli nalezen.\n", hledany_prvek);
35     }
36     return 0;
37 }
```

← Ukázka výstupu

</ Implementace pole />

{Cvičení 03 – Největší prvek dvouroz. p.}

- Necháme uživatele zadat 4 čísla.
- Nakonec vypíšeme největší prvek a vypíšeme jeho souřadnice.

```
Zadejte prvky dvourozměrného pole 2x2:  
Prvek [0][0]: 1  
Prvek [0][1]: 2  
Prvek [1][0]: 3  
Prvek [1][1]: 4  
Největší prvek v poli je: 4  
Nachází se na pozici [1][1].
```

```
1  #include <stdio.h>  
2  
3  #define RADKY_POLE 2  
4  #define SLOUPCE_POLE 2  
5  
6  int main() {  
7      int pole[RADKY_POLE][SLOUPCE_POLE];  
8      printf("Zadejte prvky dvourozmerneho pole %dx%d:\n", RADKY_POLE, SLOUPCE_POLE);  
9      for (int i = 0; i < RADKY_POLE; i++)  
10     {  
11         for (int j = 0; j < SLOUPCE_POLE; j++)  
12         {  
13             printf("Prvek [%d][%d]: ", i, j);  
14             scanf("%d", &pole[i][j]);  
15         }  
16     }  
17     int max_prvek = pole[0][0];  
18     int max_radek = 0;  
19     int max_sloupec = 0;  
20  
21     for (int i = 0; i < RADKY_POLE; i++)  
22     {  
23         for (int j = 0; j < SLOUPCE_POLE; j++)  
24         {  
25             if (pole[i][j] > max_prvek)  
26             {  
27                 max_prvek = pole[i][j];  
28                 max_radek = i;  
29                 max_sloupec = j;  
30             }  
31         }  
32     }  
33     printf("Nejvetsi prvek v poli je: %d\n", max_prvek);  
34     printf("Nachazi se na pozici [%d][%d].\n", max_radek, max_sloupec);  
35  
36     return 0;  
37 }
```

← Ukázka výstupu

</Textové řetězce />

- V jazyce C lze s řetězcí pracovat několika způsoby.
- Neexistuje zde datový typ string jako je tomu ve vyšších jazycích, z toho důvodu **používáme pole pro práci s textovými řetězci.**
- Na konci našeho pole **nesmíme zapomenout napsat nulový znak (\0)**, který slouží k zakončení textového

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char text[5] = {'T', 'E', 'S', 'T', '\0'};
6      printf("%s", text);
7      return 0;
8  }
```

1. způsob



```
1  #include <stdio.h>
2
3  int main()
4  {
5      char text[] = "TEST";
6      printf("%s", text);
7      return 0;
8  }
```

2. způsob

</Textové řetězce />

{Práce s jednotlivými znaky}

- S textovým řetězcem lze zacházet úplně stejně jako s polem.
- Na prvky můžeme opět přistupovat pomocí indexů, jako jsme to dělali doposud a zároveň můžeme přepisovat jednotlivé prvky v poli.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char text[] = "test";
6      text[0] = 'h';
7      text[3] = '\0';
8      printf("%s", text);
9      return 0;
10 }
```

Ukázkový výstup

hes

Místo 't' na začátku
vložíme znak 'h'

Místo 't' na konci
vložíme nulový znak

</Textové řetězce />

{Čtení/výpis řetězce}

- Textové řetězce můžeme jednoduše načítat/vypisovat pomocí funkce **printf()** a formátovací značky **%s**.
- Proměnnou pro řetězec založíme jako pole typu **char** a určíme si maximální velikost.
- (Pozn. U textových řetězců ve funkci **scanf()** nepíšeme **&**, jelikož s polem předáváme rovnou jeho adresu.)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Zadej své jméno: ");
6      char jmeno[21]; // Velikost 20 (řetězec) + 1 (nulový znak)
7      scanf("%20s", jmeno); // %20s je ochrana - přečte se pouze 20 znaků
8      printf("Ahoj uživateli %s, vítám te!", jmeno); // Vypis celého řetězce
9      return 0;
10 }
```

Nelze vkládat řetězce
s mezerou

Ošetření vstupu pro
vkládání s mezerou

```
scanf("%20[^\n]s", jmeno);
```

</Textové řetězce />

{Standardní funkce pro práci s řetězci}

- Pro práci s řetězci můžeme používat již předpřipravené funkce, stačí nám přidat knihovnu `#include <string.h>`.
- Předpřipravené funkce:
- `strlen()`, `strcat()`, `strcpy()`, `strchr()`, `strstr()`,
`strcmp()`

</Textové řetězce />

{Funkce strlen()}

- Pomocí funkce `strlen()` můžeme zjistit délku řetězce (jedná se o délku viditelné části bez znaku `\0`).

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      printf("%d", strlen("test")); // vrátí 4
7      return 0;
8  }
```

</Textové řetězce />

{Funkce strcat()}

- Pomocí funkce `strcat()` můžeme spojit 2 textové řetězce do jednoho. (Pozn. Je potřeba dávat pozor, aby byl v prvním řetězci dostatek místa).

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void)
5  {
6
7      char text[20] = "test";
8      strcat(text, " bude ve ctvrtek"); // uloží do pole text "bude ve ctvrtek"
9      printf("%s", text);
10
11     return 0;
12 }
```

</Textové řetězce />

{Funkce strcpy()}

- Pomocí funkce `strcpy()` můžeme naklonovat textový řetězec do jiné proměnné.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char str1[20] = "Hello World!";
7      char str2[20];
8      // Zkopírujeme str1 do str2
9      strcpy(str2, str1);
10     // Vypíšeme str2
11     printf("%s", str2);
12     return 0;
13 }
```

</ Rekapitulace />

- **1) Co je to pole?**
 - - *Pole je datová struktura sloužící k ukládání a organizaci více hodnot stejného datového typu pod jedním názvem.*
- **2) Jaké je využití pole?**
 - - *Ukládání seznamu prvků a kolekce dat, Maticové operace, Zpracování textu, Sledování statistik, Tvorba základních databází.*
- **3) Jaký je způsob uložení pole v paměti?**
 - - *Pole je v paměti uloženo za sebou, každá hodnota v paměti je uložena na indexu.*

</ Rekapitulace />

- **4) Co je to VLA?**
 - - *Pole, kde určíme velikost až za běhu programu.*
- **5) Pomocí čeho pracujeme s polem?**
 - - *Využíváme k tomu cykly typu for.*
- **6) Jak funguje dvourozměrné pole a jak se indexuje?**
 - - *Lze si ho představit jako tabulku, funguje jako klasické pole, ale má navíc jednu osu. Indexování: pole[počet řádků][počet sloupců]*

</ Zdroje />

- *ITnetwork [online]. [cit. 2024-02-08]. Dostupné z:
<https://www.itnetwork.cz/cecko/zaklady/tutorial-jazyk-c-textove-retezce>*